Introduction to Python

15 Problems	No Solutions Here	Chapter 01 Topics	Tip
Beginner to Medium	Create .py files	print, input, comments, syntax	Type every solution yourself!

Rules & Instructions

- 1. Create a folder called Chapter-01 inside your repo.
- 2. For each problem, create the .py file shown in the table below.
- 3. Write your solution in that file and run it - make sure it works.
- 4. Do NOT copy-paste. Type every single character yourself.
- 5. Stuck for more than 20 minutes? Re-read the relevant section in the Notes PDF.
- 6. Do NOT look at solutions before genuinely attempting the problem.
- 7. The problems build on each other - do them in order.
- 8. Comments in your code are encouraged - explain what you are doing.

Problem Index

#	Problem Title	Difficulty	Topic	File Name
01	Hello, Python!	Easy	print()	`p01_hello.py`
02	Personal Info Card	Easy	print() formatting	`p02_info_card.py`
03	Escape Sequence Explorer	Easy	Escape sequences	`p03_escape.py`
04	Separator & End Tricks	Easy	print() sep/end	`p04_sep_end.py`
05	Print Without Newlines	Easy	print() end param	`p05_no_newline.py`
06	User Greeting	Easy	input() + print()	`p06_greet.py`
07	Two Questions	Easy	Multiple input()	`p07_two_questions.py`
08	Comment Audit	Easy	Comments	`p08_comments.py`
09	Mad Libs Story	Medium	input() + print()	`p09_madlibs.py`

#	Problem Title	Difficulty	Topic	File Name
10	ASCII Name Banner	Medium	print() creativity	p10_banner.py
11	Receipt Printer	Medium	print() formatting	p11_receipt.py
12	Terminal Pattern	Medium	print() patterns	p12_pattern.py
13	Multi-line Art	Medium	Triple-quote strings	p13_art.py
14	Bug Hunt	Medium	Syntax / errors	p14_bughunt.py
15	Python Fact Sheet	Hard	All Ch01 concepts	p15_factsheet.py

Difficulty: **Easy** = Directly from notes - **Medium** = Needs some creativity - **Hard** = Combines multiple concepts

Section A - print() Fundamentals (Problems 01-05)

#01 Hello, Python! Easy

Write a Python program that prints the following output EXACTLY
(matching spacing and punctuation):

```
Hello, World!
Hello, Python!
I am learning to code.
```

Requirements:
- Use exactly 3 print() calls, one for each line.
- Do not use any variables.

> **HINT:** Each print() call automatically moves to the next line.

#02 Personal Info Card Easy

Print a personal info card formatted exactly like this:

```
==============================
Name   : Your Full Name
Age    : Your Age
City   : Your City
Hobby  : Your Hobby
==============================
```

Requirements:
- Replace the placeholders with YOUR real information.
- The === borders must be exactly 30 = characters each.
- All colons (:) must be vertically aligned.

> **HINT:** Count the = characters: '=' * 30 in a Python expression gives you 30 of them.

#03 Escape Sequence Explorer — Easy

Using a SINGLE print() statement for each, produce this output:

```
Line 1
Line 2
 (Note: both lines come from ONE print() call)

 Name:   Aryan     (a tab separates Name: and Aryan)

 He said "Python is great!"

 The file path is: C:\Users\Aryan\Desktop
```

Requirements:

- Each output must come from exactly 1 print() call.
- Use the correct escape sequences: \n \t \" \\

HINT: For a tab: \t. For a literal backslash: \\. For a quote inside quotes: \"

#04 Separator & End Tricks — Easy

Without using string concatenation (+), produce this output
using print() with the sep= parameter only:

```
2026-06-11
Python/is/awesome
one | two | three
ABCDE   (no spaces, no separator between letters)
```

Requirements:

- Each line must use ONE print() call with the sep= argument.
- The values passed to print() should be individual items,
  not a pre-joined string.

HINT: print('A','B','C','D','E', sep='') produces ABCDE

#05 Print Without Newlines

Easy

Using the end= parameter, make the following appear on a SINGLE line in the terminal (from multiple print() calls):

Python is fun and powerful!

Requirements:

- Use 5 separate print() calls (one word / phrase each).
- The final output must be on exactly ONE line.
- Bonus: also print a blank line after all of them using one more print() call with no arguments.

HINT: Use end=' ' to keep on the same line. The last print() needs end='\n' or default.

Section B - input() and Interaction (Problems 06-07)

#06 User Greeting — Easy

Write a program that:
1. Asks the user for their first name.
2. Asks the user for their favourite programming language.
3. Prints a personalised greeting like this:

```
>>> Sample run:
Enter your name: Aryan
Favourite language: Python
```
Hey Aryan! Great choice - Python is amazing!

Requirements:
- Use input() twice.
- Use print() with the user's input in the message.

HINT: Store the result of input() in a variable: name = input('Enter your name: ')

#07 Two Questions — Easy

Ask the user 3 questions and print a summary. Example run:

```
>>> Sample run:
```
What city do you live in? Jaipur
What is your favourite food? Biryani
What is your dream job? Software Engineer

--- Your Summary ---
You live in Jaipur.
Your favourite food is Biryani.
Your dream is to become a Software Engineer.

Requirements:
- Use 3 input() calls and store each answer in a variable.
- Print the summary using those variables.

HINT: You need 3 variables and 4 print() calls (1 for header, 3 for answers).

Section C - Comments and Syntax (Problem 08)

#08 Comment Audit

Easy

Write a program that calculates and prints the area of a rectangle.
The program must include ALL of the following comments:

1. A file-level docstring at the very top (triple quotes) that describes what the program does.
2. At least 2 single-line comments that explain your logic (not just repeat what the code says).
3. One inline comment on the print() line.
4. One line of commented-out code (disabled with #).

```
The output should be:
Area of the rectangle: 50
```

HINT: Area = length x width. Use values length=10, width=5 so output is 50.

Section D - Creative Challenges (Problems 09-13)

#09 Mad Libs Story — Medium

Create a Mad Libs (fill-in-the-blank story) program.
Ask the user for 5 words and use them to build a funny story.

```
Ask for: an animal, a place, a verb (action), an adjective,
    and a number.

>>> Sample run:
Enter an animal: elephant
Enter a place: library
... (3 more inputs)

Once upon a time, an elephant walked into a library...
(your story using all 5 words)
```

Requirements:
- The story must use ALL 5 user inputs.
- Minimum 3 sentences in the story.

> **HINT:** Plan your story template first, then decide what inputs you need to fill the blanks.

#10 ASCII Name Banner — Medium

Ask the user for their name, then print it inside a decorative
ASCII banner. Example for input 'ARYAN':

```
**********************
*                    *
*  Hello, ARYAN !    *
*                    *
**********************
```

Requirements:
- The border must be made of * characters.
- The banner must be at least 5 lines tall.
- The name must be centered visually inside the banner.
  - Bonus: make the banner width adjust to the name length.

> **HINT:** Use len(name) to get the name's length, then calculate the banner width dynamically.

#11 Receipt Printer

Medium

Simulate a shop receipt. Hard-code 4 items with prices.
Print a formatted receipt like this:

```
  ================================
PYTHON SHOP - RECEIPT
  ================================
Item            Price
--------------------------------
Python Book      Rs. 499
USB Cable        Rs. 199
Notebook         Rs. 79
Pen (2x)         Rs. 40
--------------------------------
TOTAL            Rs. 817
  ================================
Thank you for shopping with us!
```

Requirements: Columns must be aligned. Use \t or spaces.

HINT: Use print() with sep='' or f-strings for alignment. Count spaces manually.

#12 Terminal Pattern

Medium

Using only print() calls, reproduce this exact pattern:

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
* * * * *
* * * *
* * *
* *
*
```

Requirements:
- No loops (we haven't learned those yet - use print() only).
- The pattern must be a diamond shape (growing then shrinking).
- Exactly as shown above - spaces between each *.

HINT: 11 print() calls total. Middle line has 6 stars (widest point).

#13 Multi-line Art

Medium

Using a triple-quoted string (not multiple print() calls),
print a piece of ASCII art of your choice. The art must be:

- At least 8 lines tall and 20 characters wide.
- Original (you design it, not copy from the internet).
- Stored in a variable called art using triple quotes.
- Printed with a single print(art) call.

Ideas: a house, a rocket, a cat, a tree, the Python snake logo,
your name in block letters, a chess piece...

The goal is to practice triple-quoted strings and creative use
of print(). Have fun with this one.

HINT: art = ''' Your art here ''' - triple quotes preserve newlines automatically.

Section E - Challenge Problems (Problems 14-15)

#14 Bug Hunt — Medium

The code below has 6 deliberate bugs. Copy it into p14_bughunt.py, find and fix every bug, and make the program run correctly.

> **HINT:** Bugs can be: SyntaxError, NameError, wrong output, bad indentation, wrong quotes...

p14_bughunt.py - 6 bugs to find and fix

```
# Buggy program - find and fix all 6 bugs

Print('Welcome to Python!)

user_name = Input('What is your name? ')

print('Hello ' + userName + '!')

print('Name has', Len(user_name) 'chars.')

# Bug 5: docstring is not at top of file
# Docstring should be moved to line 1

# Bug 6: farewell is incorrectly commented out
#print('Goodbye ' + user_name + '!')
```

List the 6 bugs you found (write in your .py file as comments):

Bug 1: _____

Bug 2: _____

Bug 3: _____

Bug 4: _____

Bug 5: _____

Bug 6: _____

#15 Python Fact Sheet

Hard

Build an interactive Python fact sheet program that:

1. Has a docstring at the top describing what it does.
2. Asks the user for their name with input().
3. Prints a personalised header: 'Python Facts for <name>'
   with a border of = characters matching the header length.
4. Prints 5 interesting Python facts (research or recall them).
   Each fact should be numbered and have a blank line after it.
5. Uses at least 3 different print() features:
   - sep= parameter
   - end= parameter
   - escape sequences (at least \n and \t)
6. Ends with a motivational sign-off using the user's name.
7. Has meaningful comments throughout the code.

This is a synthesis problem - it tests everything in Chapter 01.

> **HINT:** Plan on paper first: what will you print? What inputs? What features will you show?

Python - Zero to Projects | Chapter 01 - Practice Set | Solutions are in Chapter-01-Solutions/ (check the repo after you finish!)